

CS 112

Introduction to Data Structures

WEEK 11

Binary Search Trees

Eric Araújo

Calvin University · Fall 2026

This Week

- 1 Guessing games and binary search
- 2 Tree vocabulary
- 3 The BST property **KEY**
- 4 insert and find
- 5 Why shape decides speed
- 6 Traversals
- 7 Height and remove

Seven Guesses

I'm thinking of a number between 1 and 128. You get seven guesses.

- Guessing at random — under a 6% chance
- Trying 1, 2, 3, ... — you'd need up to 128 guesses
- Guessing the **middle** each time — always wins, in 7

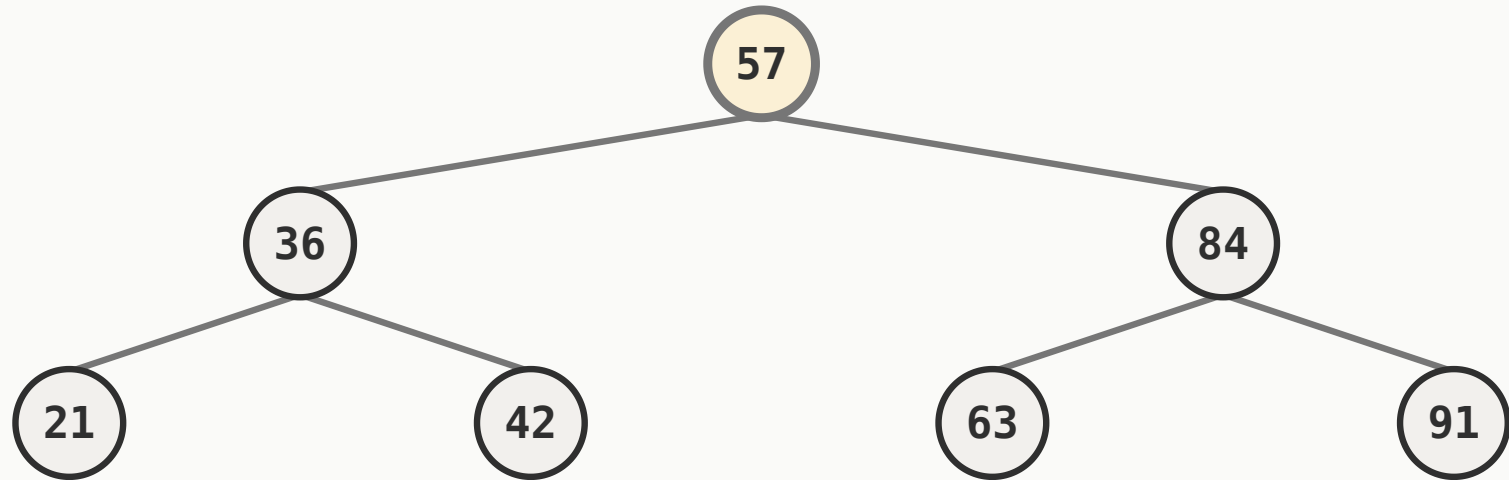
Halving the search space each time is $O(\lg n)$: $\log_2 128 = 7$. The catch is that binary search needs the data **sorted** and **indexable** — an array, not a linked list.

What If the Structure Did the Halving?

A sorted array gives fast search but slow insertion. A linked list gives fast insertion but no halving.

We want both: **every step should cut the remaining possibilities in half**, and inserting shouldn't shift anything. That structure is a binary search tree.

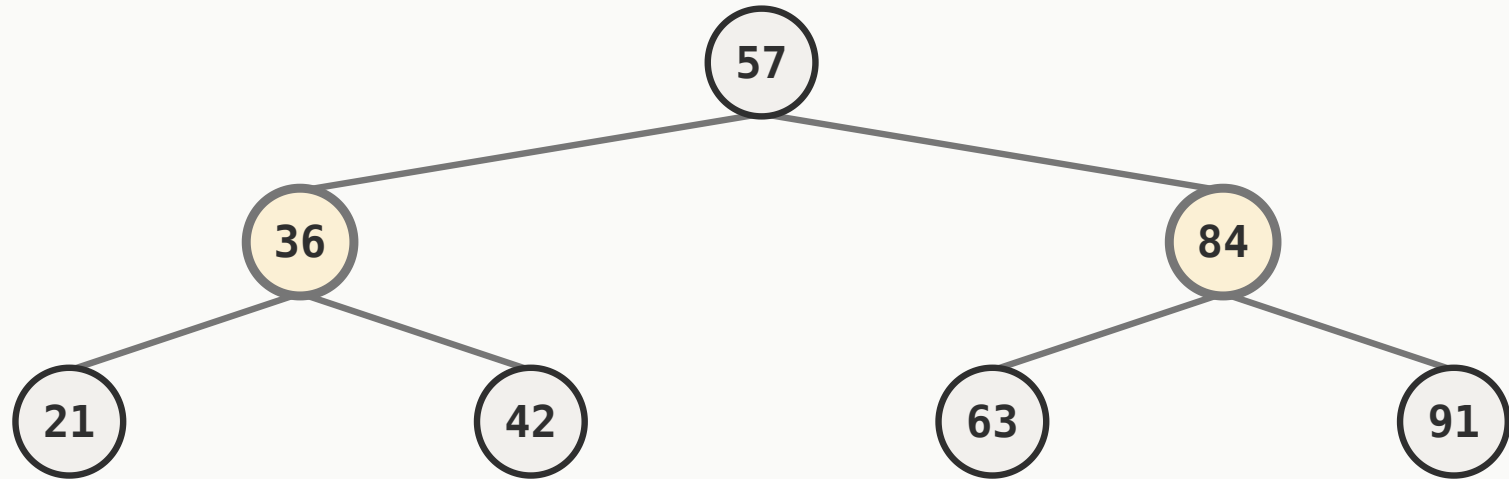
Tree Vocabulary



- **root** — the top node, no parent (57)
- **leaf** — a node with no children (21, 42, 63, 91)

- **interior** — has 1 or 2 children (36, 84)
- **height** — links from the root to the deepest node (2 here)

The BST Property



For **every** node: everything in its left subtree is smaller, everything in its right subtree is larger. Not just its children — *everything below it*.

Two Classes Again

```
class Node {  
    Item myItem;  
    Node *myLeft;  
    Node *myRight;  
};
```

```
class BST {  
    public:  
        void insert(Item it);  
        bool contains(Item it);  
    private:  
        Node *myRoot;  
        unsigned mySize;  
};
```

Same shape as the linked list: a public container hiding a private node type. A node now has **two** pointers instead of one — that's the entire structural difference.

insert

```
void Node::insert(Item it) {  
    if (it < myItem) {  
        if (myLeft == nullptr) { myLeft = new Node(it); }  
        else { myLeft->insert(it); }  
    } else {  
        if (myRight == nullptr) { myRight = new Node(it); }  
        else { myRight->insert(it); }  
    }  
}
```

Compare, go left or right, recurse. New nodes always arrive as **leaves** — which is why the Node constructor sets both children to `nullptr`.

TALK TO YOUR NEIGHBOR · TTYN

Which insertion order builds the tree on the earlier slide (57 at the root)?

A. 21, 36, 42, 57, 63, 84, 91

B. 57, 36, 84, 21, 42, 63, 91

C. 91, 84, 63, 57, 42, 36, 21

D. 36, 21, 42, 57, 84, 63, 91

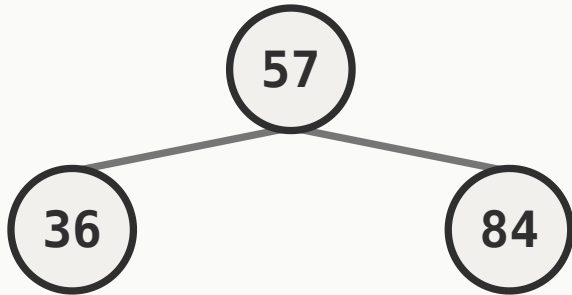
find / contains

```
bool Node::contains(Item it) const {  
    if (it == myItem) { return true; }  
    if (it < myItem) {  
        return myLeft != nullptr && myLeft->contains(it);  
    }  
    return myRight != nullptr && myRight->contains(it);  
}
```

One comparison eliminates an entire subtree — the guessing game, in code. Every step down discards roughly half of what's left.

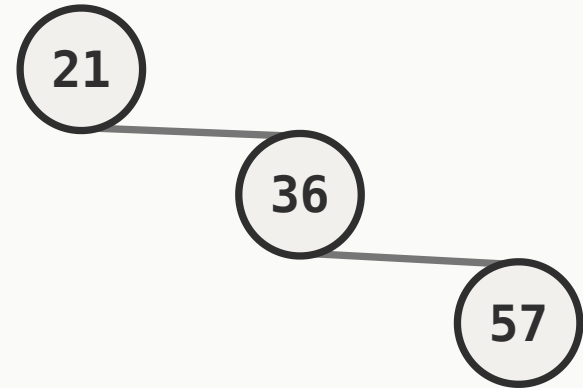
Shape Decides Everything

Balanced



height $\approx \lg n \rightarrow \mathbf{O(\lg n)}$

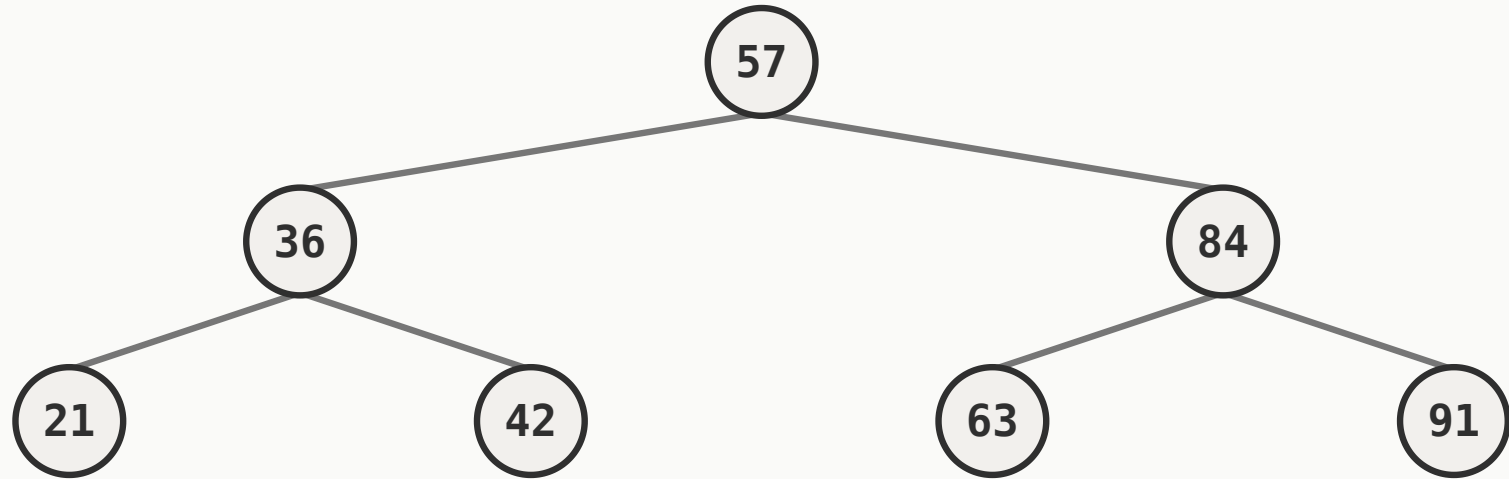
Degenerate



height = $n - 1 \rightarrow \mathbf{O(n)}$

Insert sorted data and every node goes right: you have built a linked list with extra steps. Same code, same values — the *order they arrived* ruined it. Fixing that is next week.

Traversals



Order	Rule	Result
Pre-order	visit, left, right	57 36 21 42 84 63 91
In-order	left, visit, right	21 36 42 57 63 84 91
Post-order	left, right, visit	21 42 36 63 91 84 57

In-order comes out **sorted** — that falls straight out of the BST property.

Which traversal must a BST destructor use?

- A. Pre-order — delete the root first
- B. In-order — delete in sorted order
- C. Post-order — delete both children before the node
- D. Any of them

Height

```
int Node::height() const {  
    int leftH  = (myLeft  == nullptr) ? -1 : myLeft->height();  
    int rightH = (myRight == nullptr) ? -1 : myRight->height();  
    return 1 + max(leftH, rightH);  
}
```

- An empty tree has height -1 ; a single node, 0
- Computing it costs $O(n)$ — you have to visit everything
- A **full** tree of height h holds $2^{(h+1)} - 1$ nodes

That last line is the payoff: a million nodes fit in a balanced tree of height 20. Twenty comparisons to find anything.

remove: Three Cases

- **A leaf** — unlink it from its parent and delete. Easy.
- **One child** — hand the child up to the parent. Also easy.
- **Two children** — you can't just remove it; something must take its place.

Which value can replace it without breaking the BST property? It must be larger than everything on the left and smaller than everything on the right.

TALK TO YOUR NEIGHBOR · TTYN

Removing the root 57 (children 36 and 84) — which value replaces it?

- A.** 36 — the left child
- B.** 84 — the right child
- C.** 42 — the largest value in the left subtree
- D.** 21 — the smallest value in the tree

Demo



Inserting sorted data into a BST.

(and watching it degenerate into a list)

Where BSTs Stand

Operation	Balanced BST	Degenerate BST	Sorted array
find	$O(\lg n)$	$O(n)$	$O(\lg n)$
insert	$O(\lg n)$	$O(n)$	$O(n)$
remove	$O(\lg n)$	$O(n)$	$O(n)$
in-order listing	$O(n)$	$O(n)$	$O(n)$

A balanced BST matches the sorted array on lookups and beats it badly on changes. Everything depends on that word **balanced** — which is exactly what AVL trees guarantee.

Week 11 Recap

- A BST puts smaller values left, larger right — **for every node**
- insert and find are the guessing game: compare, then discard half
- New nodes always arrive as **leaves**
- In-order traversal comes out **sorted**; the destructor must be **post-order**
- Balanced gives $O(\lg n)$; sorted input gives a list and $O(n)$
- Removing a two-child node: promote the max of the left subtree

Next: a11 →

This deck stands on earlier CS112 materials by
Joel Adams and **Victor Norman** · adapted and extended by **Eric Araújo**

